

TOMATO

Low Level Design — Complete Documentation

C++ | OOP | Design Patterns | Interview Ready

Strategy Pattern • Factory Pattern • Singleton Pattern

Class	Type	Purpose
MenuItem	Model	Ek dish ka data — name, code, price
Restaurant	Model	Restaurant + menu list (owns MenuItem)
Cart	Model	User ka shopping basket
User	Model	Customer — Cart ka owner

Order	Abstract	Common order data — base class
DeliveryOrder	Concrete	Ghar pe delivery — address chahiye
PickupOrder	Concrete	Restaurant se pickup
IPaymentStrategy	Interface	Payment ka contract
CreditCard / NetBanking / UPI	Strategy	Payment implementations
IOrderFactory	Interface	Order banane ka contract
NowOrderFactory	Factory	Abhi order banao
ScheduleOrderFactory	Factory	Baad mein order banao
RestaurantManager	Singleton	Ek hi restaurant list — poori app
OrderManager	Singleton	Ek hi order list — poori app
NotificationService	Service	User ko notify karo
Tomato	Orchestrator	Sab kuch connect karta hai

Models — Real World Objects in Code

Models represent real-world things in code. Jab tu Zomato open karta hai, restaurants dikhte hain, items dikhte hain — yeh sab models hain.

MenuItem — Ek Dish Ka Data

Sabse simple class. Koi dependency nahi. Ek dish represent karta hai.

```
class MenuItem {  
    private:  
        string code; // 'P001' – unique id  
        string name; // 'Margherita' – display name  
        int price; // 299 – rupees mein  
    public:  
        MenuItem(string code, string name, int price)  
        : code(code), name(name), price(price) {}  
  
        string getName() const { return name; }  
        string getCode() const { return code; }  
        int getPrice() const { return price; }  
};
```

DRY RUN — MenuItem banate waqt

Step 1: MenuItem m1('P001', 'Margherita', 299);

code = 'P001'

name = 'Margherita'

price = 299

Step 2: m1.getName() returns 'Margherita'

Step 3: m1.getPrice() returns 299

Key Concepts

Member Initializer List :code(code) — direct initialize, assign nahi.

const after function — yeh function object ko modify nahi karta.

private — data bahar se direct access nahi. Sirf getters se.

Restaurant — Restaurant Ka Data

Restaurant apna menu khud rakhta hai. MenuItem objects ki list. Yeh COMPOSITION hai — Restaurant owns its menu.

```
class Restaurant {
private:
    int id;
    string name;
    string location;
    vector<MenuItem> menu; // owns menu items – composition
public:
    Restaurant(int id, string name, string location)
        : id(id), name(name), location(location) {}

    void addMenuItem(MenuItem item) {
        menu.push_back(item);
    }

    int getId() const { return id; }
    string getName() const { return name; }
    string getLocation() const { return location; }
    vector<MenuItem> getMenu() const { return menu; }
```

```
};
```

DRY RUN — Restaurant setup

```
Restaurant r1(1, 'Pizza Hut', 'Ahmedabad');  
  
id=1 name='Pizza Hut' location='Ahmedabad' menu=[]  
  
r1.addMenuItem(MenuItem('P001', 'Margherita', 299));  
  
menu = [Margherita(299)]  
  
r1.addMenuItem(MenuItem('P002', 'Pepperoni', 499));  
  
menu = [Margherita(299), Pepperoni(499)]  
  
r1.getMenu().size() returns 2
```

Composition vs Association

Composition (filled diamond)

```
vector<MenuItem> menu; // Restaurant OWNS MenuItem
```

Jab Restaurant destroy hoga, MenuItem bhi destroy hoga!

Association (open diamond)

```
Restaurant* restaurant; // Cart sirf POINTS TO Restaurant
```

Cart destroy ho — Restaurant safe rehta hai!

Cart — User Ka Shopping Basket

```
class Cart {  
    private:
```

```

vector<MenuItem> items;

Restaurant* restaurant; // POINTER – cart owns nahi karta!

public:

Cart() : restaurant(nullptr) {}

void setRestaurant(Restaurant* r) { restaurant = r; }

void addToCart(MenuItem item) { items.push_back(item); }

int totalCost() const {
    int total = 0;
    for (auto item : items) total += item.getPrice();
    return total;
}

bool isEmpty() const { return items.empty(); }

Restaurant* getRestaurant() const { return restaurant; }

vector<MenuItem> getItems() const { return items; }

};

```

DRY RUN – Cart use karna

Restaurant r1(1, 'Pizza Hut', 'Ahmedabad');

Cart cart;

items=[] restaurant=nullptr

cart.setRestaurant(&r1);

restaurant = address of r1 (r1 copy nahi hua!)

cart.addToCart(MenuItem('P001', 'Margherita', 299));

items = [Margherita(299)]

cart.totalCost() returns 299

cart.isEmpty() returns false

Pointer aur & ka simple rule

Restaurant* r; // * = address store karega

cart.setRestaurant(&r1); // & = r1 ka address do

Dost ka phone number analogy:

Phone = Cart

Number = Pointer (*)

Dost = Restaurant

Phone ne dost ko banaya nahi — sirf number store kiya!

User — Customer

```
class User {  
    private:  
        int id;  
        string name;  
        string address;  
        Cart cart; // by VALUE – User OWNS cart  
    public:  
        User(int id, string name, string address)  
        : id(id), name(name), address(address) {}  
  
        Cart& getCart() { return cart; } // & = reference – original!  
  
        int getId() const { return id; }  
        string getName() const { return name; }  
        string getAddress() const { return address; }  
};
```

Cart& kyun — Reference ka importance

Bina reference (GALAT):

```
Cart getCart() { return cart; } // COPY ban jata!
```

```
user.getCart().addToCart(item); // COPY mein add — original unchanged!
```

Reference ke saath (SAHI):

```
Cart& getCart() { return cart; } // ORIGINAL milta hai
```

```
user.getCart().addToCart(item); // ORIGINAL mein add!
```


Strategy Pattern — Payment System

Strategy Pattern: define a family of algorithms, put each in a separate class, make their objects interchangeable.

Problem — Bina Strategy Pattern ke

Order class mein yeh likhna padta:

```
if (payment == 'UPI') { /* UPI code */ }
```

```
else if (payment == 'CARD') { /* Card code */ }
```

```
else if (payment == 'NET') { /* Net code */ }
```

Kal PhonePe aaya => Order class kholni + badalni padegi!

OPEN/CLOSED PRINCIPLE VIOLATE! Bura design!

Solution — Strategy Pattern ke saath

Order class mein sirf itna:

```
strategy->pay(amount);
```

Kal PhonePe aaya => sirf nayi class banao!

Order class touch mat karo!

OPEN FOR EXTENSION, CLOSED FOR MODIFICATION!

IPaymentStrategy — Interface (Contract)

```
class IPaymentStrategy {
public:
    virtual void pay(int amount) = 0; // pure virtual – MUST implement
    virtual ~IPaymentStrategy() = default;
};
```

'= 0' matlab pure virtual. IPaymentStrategy ka object nahi ban sakta. Yeh sirf ek CONTRACT hai — jo bhi payment method banega, pay() implement karna PADEGA.

Implementations — CreditCard, NetBanking, UPI

```
class CreditCard : public IPaymentStrategy {
public:
    void pay(int amount) override {
        cout << 'Paid Rs.' << amount << ' using Credit Card!' << endl;
    }
};

class NetBanking : public IPaymentStrategy {
public:
    void pay(int amount) override {
        cout << 'Paid Rs.' << amount << ' using Net Banking!' << endl;
    }
};

class UPI : public IPaymentStrategy {
public:
    void pay(int amount) override {
        cout << 'Paid Rs.' << amount << ' using UPI!' << endl;
    }
};
```

DRY RUN — Strategy in action

```
IPaymentStrategy* p = new UPI();
```

```
p->pay(500); => Paid Rs.500 using UPI!
```

```
IPaymentStrategy* p = new CreditCard();
```

```
p->pay(500); => Paid Rs.500 using Credit Card!
```

Same line `p->pay(500)` — alag alag output! This is POLYMORPHISM!

Keyword	Explanation
virtual	Child class override kar sakti hai
= 0	Pure virtual — object nahi ban sakta, sirf inherit
override	Compiler check karta hai — parent mein hai ya nahi
IPaymentStrategy*	Base pointer — koi bhi payment strategy rakh sako

Order Classes — Inheritance in Action

Order ek abstract base class hai. DeliveryOrder aur PickupOrder usse inherit karte hain. Common cheezein ek jagah — alag cheezein alag jagah. DRY Principle!

Why Abstract Base Class?

Problem	Solution
Bina Base Class (GALAT)	Base Class ke saath (SAHI)
DeliveryOrder mein:	Order mein ek baar:
id — repeat!	id, user, restaurant
user — repeat!	items, strategy
restaurant — repeat!	pay()
items — repeat!	DeliveryOrder mein SIRF:
pay() — repeat!	deliveryAddress

Order — Abstract Base Class

```
class Order {  
    protected:  
    int id;  
    User user;  
    Restaurant* restaurant;
```

```

vector<MenuItem> items;
IPaymentStrategy* strategy;

public:
Order(int id, User user, Restaurant* rest)
: id(id), user(user), restaurant(rest), strategy(nullptr) {}

void setPaymentStrategy(IPaymentStrategy* s) { strategy = s; }
void addItem(MenuItem item) { items.push_back(item); }

void pay() {
if (strategy == nullptr) {
cout << 'No Payment Selected!' << endl; return;
}
int total = 0;
for (auto item : items) total += item.getPrice();
strategy->pay(total); // delegate to strategy!
}

int getId() const { return id; }
User getUser() const { return user; }
Restaurant* getRestaurant() const { return restaurant; }
vector<MenuItem> getItems() const { return items; }

virtual string getType() const = 0;
virtual ~Order() = default;
};

```

private vs protected vs public

private — sirf apni class access kar sakti. Child NAHI.

protected — apni class + child classes access kar sakti.

public — sab access kar sakte hain.

Order mein protected isliye — DeliveryOrder/PickupOrder

directly id, user etc access kar sakein.

DRY RUN — pay() function

```
order.addItem(Menuitem('P001', 'Margherita', 299));  
  
order.addItem(Menuitem('P002', 'Pepperoni', 499));  
  
order.setPaymentStrategy(new UPI());  
  
order.pay();
```

Step 1: strategy nullptr? => nahi, UPI hai

Step 2: total = 0

Step 3: Margherita => total = 299

Step 4: Pepperoni => total = 798

Step 5: strategy->pay(798) => Paid Rs.798 using UPI!

DeliveryOrder and PickupOrder

```
class DeliveryOrder : public Order {  
private:  
    string deliveryAddress;  
public:  
    DeliveryOrder(int id, User user, Restaurant* rest, string address)  
        : Order(id, user, rest), // Step 1: parent constructor  
        deliveryAddress(address) // Step 2: own member  
    {}  
    string getType() const override { return 'DELIVERY'; }  
    string getDeliveryAddress() const { return deliveryAddress; }  
};
```

```

class PickupOrder : public Order {
private:
    string resAddress;
public:
    PickupOrder(int id, User user, Restaurant* rest, string address)
        : Order(id, user, rest),
          resAddress(address)
    {}
    string getType() const override { return 'PICKUP'; }
    string getResAddress() const { return resAddress; }
};

```

DRY RUN — DeliveryOrder banate waqt

DeliveryOrder d(1, user, &r1, 'Navrangpura');

Step 1: Order(1, user, &r1) chala

id = 1

user = user

restaurant = &r1

strategy = nullptr

Step 2: deliveryAddress = 'Navrangpura'

d.getType() => 'DELIVERY'

d.getDeliveryAddress() => 'Navrangpura'

d.getId() => 1 (inherited from Order!)

DeliveryOrder

PickupOrder

Extra field	deliveryAddress	resAddress
getType()	'DELIVERY'	'PICKUP'
Inherits	Order	Order
Common data	id, user, rest, items, strategy	same

Factory Pattern — Order Creation

Factory Pattern: object banane ki responsibility factory ko do. Client code ko nahi pata — andar kya bana!

Problem — Bina Factory ke

main() mein har jagah yeh likhna padta:

```
if (type == 'DELIVERY') {  
  
    DeliveryOrder* d = new DeliveryOrder(...);  
  
} else {  
  
    PickupOrder* p = new PickupOrder(...);  
  
}
```

Har jagah yeh if-else repeat! Bura code!

Solution — Factory ke saath

main() mein sirf itna:

```
Order* order = factory->createOrder(1, user, &restaurant);
```

Factory andar decide karegi — Delivery ya Pickup!

main() ko pata bhi nahi!

IOrderFactory — Interface

```
class IOrderFactory {
```

```
public:
virtual Order* createOrder(int id, User user, Restaurant* rest) = 0;
virtual ~IOrderFactory() = default;
};
```

NowOrderFactory — Abhi Order Karo

```
class NowOrderFactory : public IOrderFactory {
public:
Order* createOrder(int id, User user, Restaurant* rest) override {
string type, address;
cout << 'Delivery ya Pickup? (D/P): ';
cin >> type;
if (type == 'D') {
cout << 'Delivery address do: '; cin >> address;
return new DeliveryOrder(id, user, rest, address);
} else {
cout << 'Pickup address do: '; cin >> address;
return new PickupOrder(id, user, rest, address);
}
}
};
```

ScheduleOrderFactory — Baad Mein Order Karo

```
class ScheduleOrderFactory : public IOrderFactory {
private:
string scheduleTime; // extra – sirf scheduled orders mein!
public:
ScheduleOrderFactory(string time) : scheduleTime(time) {}

Order* createOrder(int id, User user, Restaurant* rest) override {
```

```

string type, address;

cout << 'Scheduled for: ' << scheduleTime << endl;
cout << 'Delivery ya Pickup? (D/P): '; cin >> type;
if (type == 'D') {
    cout << 'Delivery address do: '; cin >> address;
    return new DeliveryOrder(id, user, rest, address);
} else {
    cout << 'Pickup address do: '; cin >> address;
    return new PickupOrder(id, user, rest, address);
}
}
};

```

DRY RUN — Factory Pattern

`IOrderFactory* factory = new NowOrderFactory();`

`Order* order = factory->createOrder(1, user, &r1);`

Console: Delivery ya Pickup? (D/P):

User: D

Console: Delivery address do:

User: Navrangpura

=> `return new DeliveryOrder(1, user, &r1, 'Navrangpura')`

=> order points to DeliveryOrder object!

`order->getType() => 'DELIVERY'`

	NowOrderFactory	ScheduleOrderFactory
Extra field	None	scheduleTime

Constructor	Default	Takes time string
createOrder()	D/P choose karo	Shows time first, then D/P
Use case	Order karo abhi	Order schedule karo

Singleton Pattern — One Instance Only

Singleton: class ka sirf ek hi instance hona chahiye poori app mein. Ek hi manager — poori app mein!

Real life analogy

Poore Zomato app mein restaurants ki ek hi list hogi!

```
RestaurantManager rm1; // alag list
```

```
RestaurantManager rm2; // alag list — PROBLEM!
```

```
rm1.addRestaurant(pizzaHut);
```

```
rm2.getRestaurants(); // pizzaHut nahi milega! BUGGY!
```

Singleton se: har jagah same instance — same data!

Singleton ke 4 Rules

Rule	Code	Kyun
Private Constructor	<code>RestaurantManager() {}</code>	Bahar se new nahi kar sakte!
Copy Delete karo	<code>(const RM&) = delete</code>	Duplicate nahi ban sakta!
Static Instance	<code>static RM* instance</code>	Ek hi object poori app mein!

getInstance()	static RM* getInstance()	Global access point — sabka rasta!
---------------	--------------------------	---------------------------------------

RestaurantManager — Singleton

```

class RestaurantManager {
private:
    RestaurantManager() {}
    RestaurantManager(const RestaurantManager&) = delete;
    RestaurantManager& operator=(const RestaurantManager&) = delete;

    static RestaurantManager* instance;
    vector<Restaurant> restaurants;

public:
    static RestaurantManager* getInstance() {
        if (instance == nullptr) {
            instance = new RestaurantManager();
        }
        return instance;
    }

    void addRestaurant(Restaurant r) { restaurants.push_back(r); }

    vector<Restaurant> searchByLoc(string loc) {
        vector<Restaurant> result;
        for (auto r : restaurants)
            if (r.getLocation() == loc) result.push_back(r);
        return result;
    }
};

RestaurantManager* RestaurantManager::instance = nullptr;

```

DRY RUN — getInstance()

First call:

RestaurantManager::getInstance()

=> instance == nullptr? YES

=> instance = new RestaurantManager()

=> return instance (address: 0x1234)

Second call:

RestaurantManager::getInstance()

=> instance == nullptr? NO

=> return instance (same address: 0x1234!)

Hamesha same instance! Hamesha same data!

OrderManager — Singleton

```
class OrderManager {
private:
    OrderManager() {}
    OrderManager(const OrderManager&) = delete;
    OrderManager& operator=(const OrderManager&) = delete;

    static OrderManager* instance;
    vector<Order*> orders; // Order* – kyunki Order abstract hai!

public:
    static OrderManager* getInstance() {
        if (instance == nullptr) instance = new OrderManager();
        return instance;
    }
}
```

```

void addOrder(Order* order) { orders.push_back(order); }

void listOrders() const {
    if (orders.empty()) { cout << 'Koi order nahi!' << endl; return; }
    for (auto order : orders) {
        cout << '-----' << endl;
        cout << 'Order ID : ' << order->getId() << endl;
        cout << 'Type : ' << order->getType() << endl;
        cout << 'User : ' << order->getUser().getName() << endl;
    }
}

};

OrderManager* OrderManager::instance = nullptr;

```

vector vs vector — Kyun alag?

RestaurantManager: vector<Restaurant>

Restaurant abstract nahi — value store kar sakte hain!

OrderManager: vector<Order*>

Order ABSTRACT hai — value store nahi hoti!

vector<Order> orders; // ERROR! abstract class!

vector<Order*> orders; // CORRECT! pointer store karo!

Rule: Abstract class? => Pointer use karo!

NotificationService + Tomato — All Together

NotificationService

```
class NotificationService {
private:
    Order* order; // pointer – sirf padhna hai, own nahi karna
public:
    NotificationService(Order* o) : order(o) {}

    void notifyUser() const {
        cout << '=====' << endl;
        cout << ' ORDER CONFIRMATION! ' << endl;
        cout << '=====' << endl;
        cout << 'Hey ' << order->getUser().getName() << '!' << endl;
        cout << 'Order Type : ' << order->getType() << endl;
        cout << 'Order ID : ' << order->getId() << endl;
        cout << 'Restaurant : ' << order->getRestaurant()->getName() << endl;
        cout << 'Items : ' << order->getItems().size() << endl;
        cout << '=====' << endl;
    }
};
```

Arrow (->) vs Dot (.) operator

order->getUser() // order is POINTER => arrow (->)

order->getUser().getName() // getUser() returns VALUE => dot (.)

```
order->getRestaurant()->getName() // getRestaurant() returns POINTER  
=> arrow
```

Rule: Pointer => Arrow (->) | Value => Dot (.)

Tomato — Orchestration Class (Main Flow)

```
class Tomato {  
    public:  
    void run() {  
        setupRestaurants();  
        User user(1, 'Rahul', 'Ahmedabad');  
  
        vector<Restaurant> list =  
            RestaurantManager::getInstance()->searchByLoc('Ahmedabad');  
        // show + choose restaurant ...  
        // show menu + choose item ...  
        user.getCart().setRestaurant(&selected);  
        user.getCart().addToCart(menu[iChoice]);  
  
        // Factory  
        IOrderFactory* factory = new NowOrderFactory();  
        Order* order = factory->createOrder(1, user, &selected);  
        for (auto item : user.getCart().getItems()) order->addItem(item);  
  
        // Payment  
        IPaymentStrategy* payment = new UPI();  
        order->setPaymentStrategy(payment);  
        order->pay();  
  
        // Save + Notify  
        OrderManager::getInstance()->addOrder(order);  
        NotificationService notification(order);  
        notification.notifyUser();  
    }  
};
```

```
OrderManager::getInstance()->listOrders();

delete factory; delete payment;
}

private:
void setupRestaurants() {
    Restaurant r1(1, 'Pizza Hut', 'Ahmedabad');
    r1.addMenuItem(MenuItem('P001', 'Margherita', 299));
    r1.addMenuItem(MenuItem('P002', 'Pepperoni', 499));
    RestaurantManager::getInstance()->addRestaurant(r1);
}
};

int main() { Tomato app; app.run(); return 0; }
```

Complete System Dry Run

Poora flow ek baar — start se end tak. User Burger King se Veg Burger order karta hai, UPI se pay karta hai, Pune mein delivery chahiye.

Step	Code	Result
1. App start	Tomato app; app.run();	Program shuru
2. Restaurants	setupRestaurants()	Pizza Hut + Burger King added
3. User	User user(1,'Rahul','Ahmedabad')	user ready, cart empty
4. Search	searchByLoc('Ahmedabad')	list = [Pizza Hut, Burger King]
5. Select	rChoice = 1	selected = Burger King
6. Menu	selected.getMenu()	[Whopper, Veg Burger]
7. Cart	iChoice = 1	Veg Burger(199) cart mein
8. Factory	new NowOrderFactory()	factory ready
9. Order type	User types: D	DeliveryOrder(..., 'Pune')
10. Items	order->addItem(VegBurger)	order.items = [VegBurger]
11. Payment	new UPI()	strategy = UPI
12. Pay	order->pay()	Paid Rs.199 using UPI!
13. Save	OrderManager::getInstance()->addOrder(order)	orders = [order1]

14. Notify	notification.notifyUser()	ORDER CONFIRMATION shown
15. List	listOrders()	Order ID:1 DELIVERY Rahul

Last Minute Revision — Interview Ready!

3 Design Patterns — Quick Reference

Pattern	Used For	Key Classes	Interview Line
Strategy	Payment method change karo runtime pe	IPaymentStrategy , UPI, Card, NetBanking	Define family of algos, make interchangeable
Factory	Object creation hide karo	IOrderFactory, NowFactory, ScheduleFactory	Object banane ki responsibility factory ko do
Singleton	Ek hi instance chahiye	RestaurantManager, OrderManager	One instance, global access point

Class Relationships

Relationship	Symbol	Example	Meaning
Composition	Filled Diamond	Restaurant has MenuItem	Owns — ek saath destroy
Association	Open Diamond	Cart points to Restaurant	Points to — independent
Inheritance	Arrow	DeliveryOrder is-a Order	Child extends parent

Realization	Dashed Arrow	CreditCard implements IPaymentStrategy	Interface implement karo
-------------	--------------	--	-----------------------------

Common Interview Questions

Q: Strategy vs Factory pattern mein difference?

Strategy: BEHAVIOR change karta hai runtime pe (payment method)

Factory: OBJECT CREATION hide karta hai (order type)

Strategy = 'Kaise karna hai' decide karo

Factory = 'Kya banana hai' decide karo

Q: Singleton thread safe nahi — kaise banayein?

```
static RestaurantManager* getInstance() {
    static RestaurantManager instance; // C++11 — thread safe!
    return &instance;
}
```

C++11 mein static local variable thread safe hoti hai!

Q: Abstract class aur interface mein fark?

Interface (C++ mein): sirf pure virtual functions, koi data nahi

```
class IPaymentStrategy { virtual void pay() = 0; };
```

Abstract class: kuch concrete methods bhi + data

```
class Order { void pay(){...} virtual string getType()=0; };
```

Tomato mein dono hain!

Q: new ke baad delete kyun?

new = heap pe memory allocate karo

delete = heap ki memory free karo

delete factory; // NowOrderFactory free

delete payment; // UPI/CreditCard free

Bina delete ke => Memory Leak!

Pointer Cheatsheet

Situation	Use	Example
Declaration	Type*	Restaurant* r;
Address lena	&	cart.setRestaurant(&r1);
Access members	->	order->getId()
Value access	.	user.getName()
Empty pointer	nullptr	strategy = nullptr;
Heap alloc	new	new DeliveryOrder(...)
Memory free	delete	delete factory;

One Line Summary — Every Class

Class	One Line
-------	----------

MenuItem	Ek dish — code, name, price
Restaurant	Restaurant ka data + menu (owns MenuItem)
Cart	User ka basket — items + restaurant pointer
User	Customer — owns Cart
IPaymentStrategy	Payment ka contract — pay() implement karo
CreditCard/Net/UPI	Payment strategies — pay() override karo
Order	Abstract base — common order data + pay()
DeliveryOrder	Order + delivery address, getType()=DELIVERY
PickupOrder	Order + restaurant address, getType()=PICKUP
IOrderFactory	Order banane ka contract — createOrder()
NowOrderFactory	Abhi order banao — D ya P choose karo
ScheduleOrderFactory	Baad mein order — scheduleTime extra
RestaurantManager	Singleton — ek hi restaurant list
OrderManager	Singleton — ek hi order list (Order* store)
NotificationService	Order details lekar user ko notify karo
Tomato	Sab kuch connect karta hai — orchestrator

**ALL THE BEST FOR YOUR INTERVIEW! — Tomato
LLD Complete**